

Advanced Topics



Daftar Isi



Virtual Machines

- Overview & History
- Benefits and Features
- Building Blocks
- Types of VMs (Type 1 & Type 2)
- Implementations
- OS Components Integration
- Examples & Use Cases
- Virtualization Research
- Summary



Networks & Distributed Systems

- Advantages of Distributed Systems
- Network & Communication Structure
- Distributed Operating Systems
- Design Issues (2 parts)
- Distributed File Systems (2 parts)
- DFS Naming & Transparency
- Remote File Access
- Final Thoughts
- Summary

Apa itu Virtual Machine?

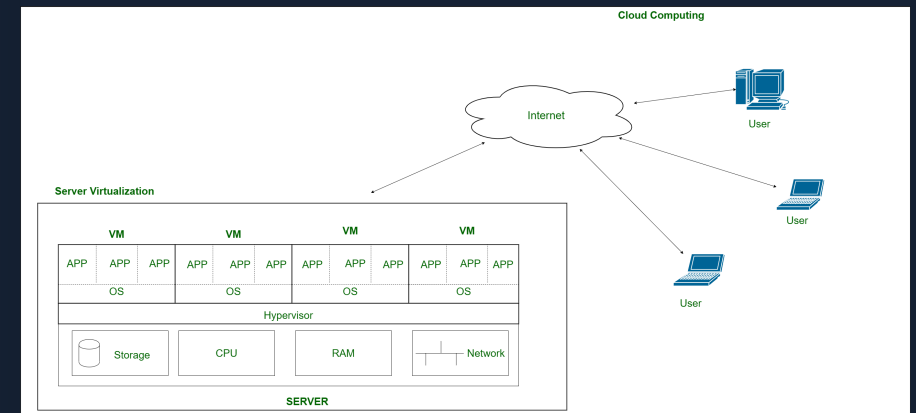
Software yang berjalan seperti **komputer fisik** dengan sistem operasi dan aplikasinya sendiri

Konsep Dasar

- Mensimulasikan komputer dalam komputer
- Menggunakan **hardware fisik** sebagai sumber daya
- Berjalan **independen** dari host

Kerjanya Bagaimana?

Hypervisor mengatur sumber daya hardware sehingga beberapa OS bisa berjalan bersamaan pada satu mesin fisik



1960s

Awal Virtualisasi

IBM System/360 dan VM/370 memulai era virtualisasi mainframe

1990s

x86 Virtualisasi

VMware mempopulerkan virtualisasi pada komputer personal dan server

2010s

Container Era

Docker dan Kubernetes memperkenalkan teknologi container yang lebih ringan

1970s–1980s

Mainframe Era

VM/370 dan produk IBM lainnya dominasi di lingkungan enterprise

2000s

Cloud Revolution

AWS, Azure, Google Cloud memanfaatkan virtualisasi untuk cloud computing

Modern

Era Saat Ini

Serverless, multi-cloud, dan hybrid cloud menjadi standar industri

Efisiensi Hardware

Satu server fisik bisa menjalankan beberapa VM sekaligus, mengurangi pembelian hardware baru

Portabilitas Mudah

VM bisa dipindahkan antar server tanpa instal ulang. Sangat fleksibel untuk migrasi

Konsolidasi Server

Menggabungkan beberapa server fisik menjadi lebih sedikit, menghemat ruang dan biaya operasional

Isolasi Sistem

Setiap VM terisolasi satu sama lain. Jika satu VM crash, VM lain tetap aman

Backup & Recovery

Snapshot VM memungkinkan backup cepat dan recovery instan dalam hitungan menit

Development & Testing

Lingkungan testing yang terisolasi, memudahkan uji coba aplikasi tanpa mempengaruhi produksi

Virtual Machines / Building Blocks

Hardware Fisik

- CPU (Prosesor)
- Memory (RAM)
- Storage (Disk)
- Network Adapter

Hypervisor

- Software pengatur virtualisasi
- Mengalokasikan hardware ke VM
- Manajemen sumber daya

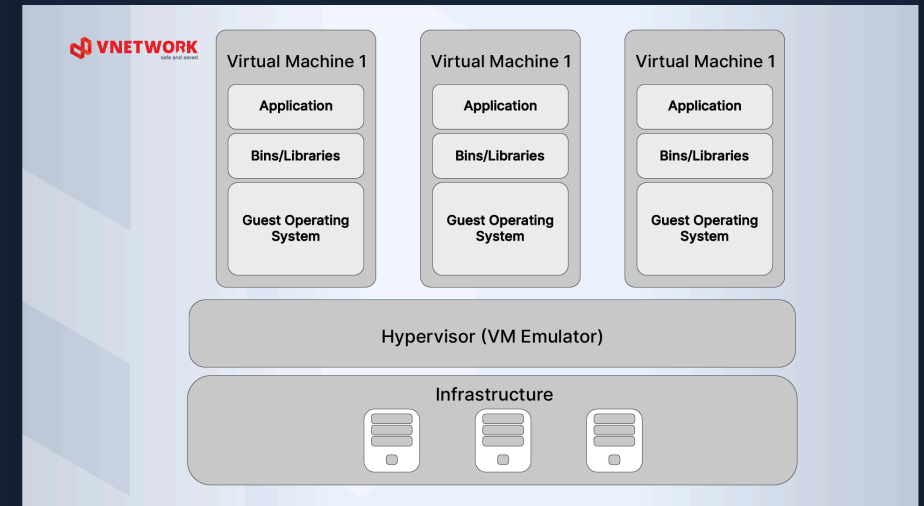
Guest OS

- Sistem operasi dalam VM
- Windows, Linux, macOS
- Berjalan independently

Applications

- Program dalam Guest OS
- Software & layanan
- Isolasi antar aplikasi

 Hypervisor bertindak sebagai "jembatan" antara hardware fisik dan sistem operasi virtual



 VM Layers

Type 1 Hypervisor

Bare Metal

- Berjalan langsung di hardware fisik
- Performa lebih tinggi
- Tidak memerlukan OS host

Contoh:

VMware ESXi, Microsoft Hyper-V, KVM

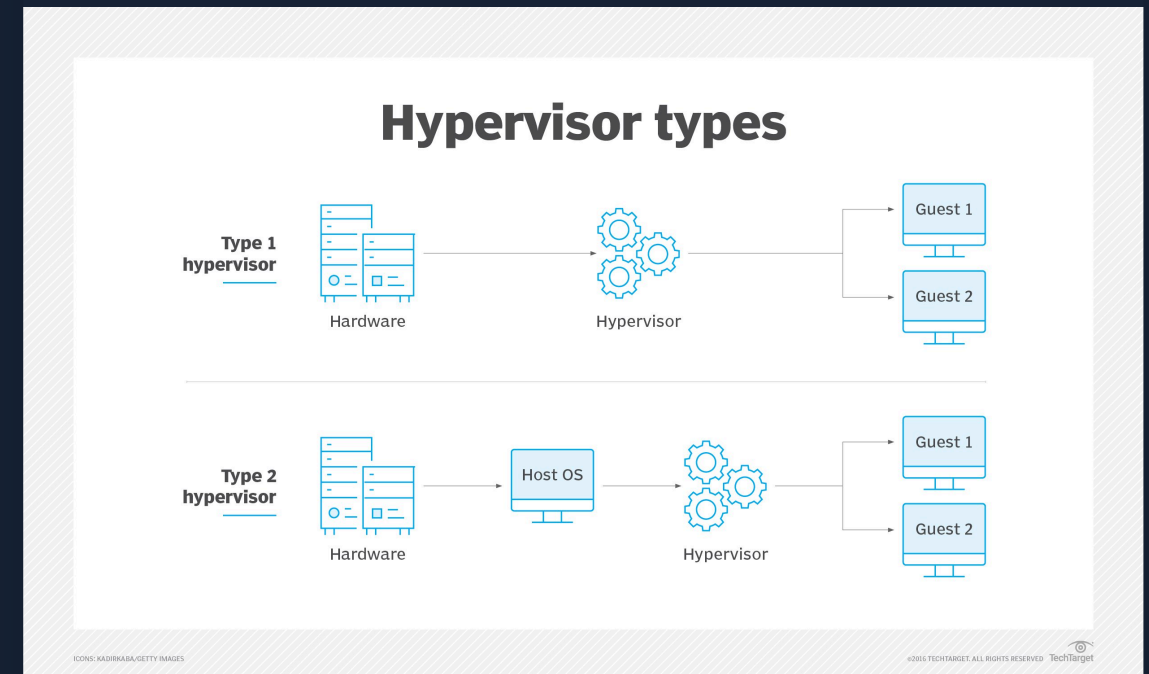
Type 2 Hypervisor

Hosted

- Berjalan di atas OS host
- Mudah diinstall & digunakan
- Cocok untuk pengembangan

Contoh:

VirtualBox, VMware Workstation, Parallels Desktop



 Hypervisor Architecture

Virtual Machines / Types of VMs (Part 2)

VMware Workstation

- Desktop virtualization
- Windows, Linux, macOS
- Powerful untuk pengembangan
- Integrasi tinggi dengan host

Ideal: Developer testing & development

Oracle VirtualBox

- Open-source & gratis
- Cross-platform
- Mendukung banyak OS guest
- Community aktif

Ideal: Personal learning & experimentation

Microsoft Hyper-V

- Type 1 hypervisor
- Built-in di Windows Server
- Enterprise-ready
- Integrasi dengan ekosistem Microsoft

Ideal: Server virtualization & enterprise

KVM (Kernel-based VM)

- Type 2 hypervisor (kernel module)
- Open-source (Linux)
- Performa tinggi
- Scalable untuk enterprise

Ideal: Cloud providers & large-scale deployment

Docker (Container)

- Container-based
- Share host kernel
- Lebih lightweight
- Portabilitas ekstrem

Ideal: Microservices & cloud-native applications

Virtual Machines / Virtualization & OS Components (Part 1)

CPU Virtualization

Alokasi Sumber Daya

Hypervisor membagi CPU fisik ke beberapa VM

Scheduling

Mengatur waktu eksekusi setiap VM

Hardware Assist

Intel VT-x, AMD-V mempercepat virtualisasi

Setiap VM berpikir memiliki CPU sendiri

Memory Virtualization

Virtual Address Space

Setiap VM memiliki memori virtualnya sendiri

Shadow Page Tables

Mengubah alamat virtual ke fisik secara transparan

Memory Ballooning

Mengalokasikan memori fleksibel antar VM

Memori fisik dibagi secara dinamis

I/O Virtualization

Device Access

VM mengakses perangkat keras melalui hypervisor

Paravirtualization

Guest OS modified untuk performa lebih baik

Full Virtualization

Guest OS tidak perlu dimodifikasi

Akses hardware melalui abstraksi layer

Scheduling

CPU Scheduling

Mengatur waktu eksekusi VM

Preemptive

Bisa diganggu kapan saja

Fair-Share

Pembagian adil resource

Resource Management

Dynamic Allocation

Alokasi resource fleksibel

Overcommitment

Resource bisa melampaui kapasitas

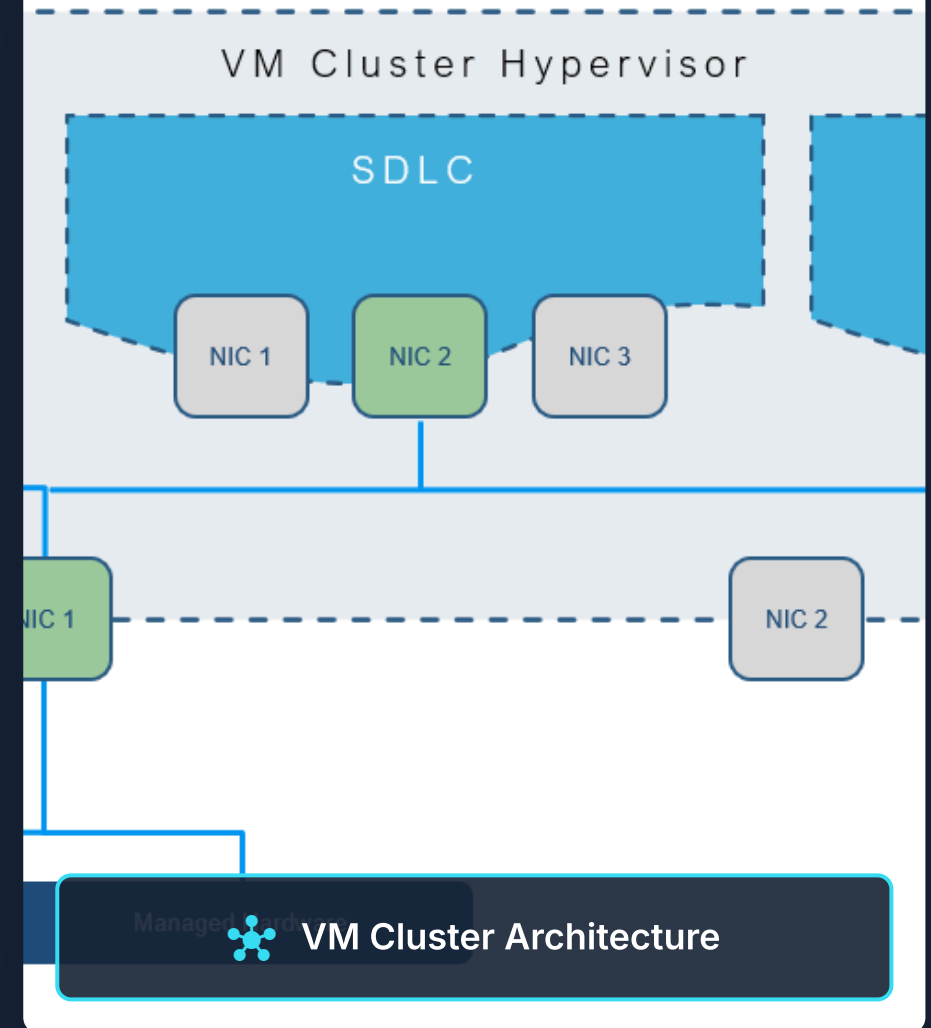
Live Migration

Pindah VM tanpa downtime

Power Management

Efisiensi penggunaan daya

Basic Install Simplified Diagram



Virtual Machines / Examples (Part 1)

Cloud Computing

IaaS (Infrastructure as a Service)

AWS, Azure, Google Cloud menyediakan VM sebagai layanan

Scalability

Skala sesuai kebutuhan bisnis

Pay-as-you-go

Bayar sesuai penggunaan resource

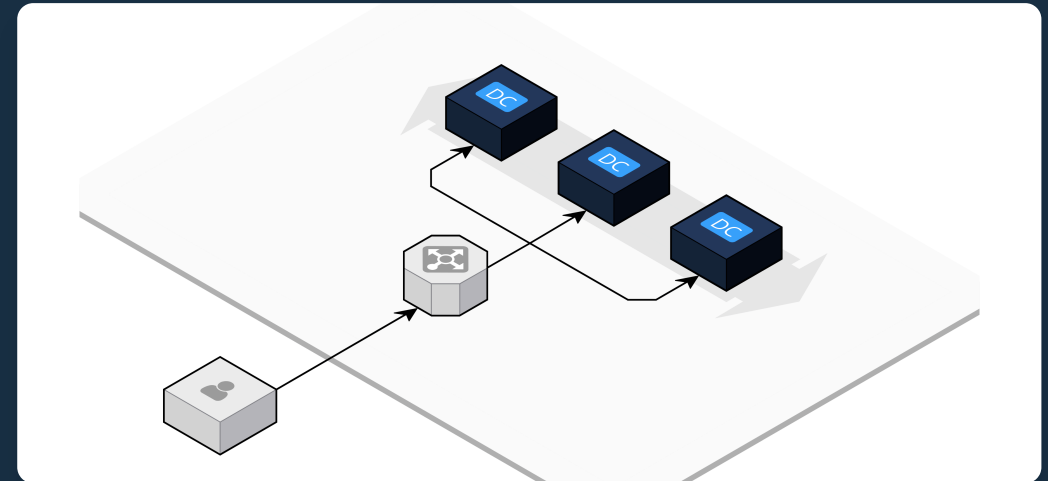


AWS

Azure

GCP

Desktop Virtualization



VDI (Virtual Desktop Infrastructure)

Desktop virtual yang terpusat

Akses dari Mana Saja

Perangkat apa saja, kapan saja

Manajemen Terpusat

Update dan maintenance lebih mudah

Testing Environment

Multi-Platform Testing

Jalankan berbagai OS dan aplikasi dalam VM untuk pengujian

Sandboxing

Isolasi aplikasi berisiko dari sistem produksi

Isolasi Total

Sistem pengujian terpisah dari sistem utama



Development



QA Testing



Sandbox

Disaster Recovery

Regular Backup

Backup VM secara rutin dan otomatis

Fast Restore

Restore cepat setelah kegagalan sistem

Multi-Site Replication

Replikasi antar lokasi untuk redundansi

Use Cases

Hosting

Enterprise

Dev Teams

Tren Penelitian Terkini

Live Migration Improvements

Migrasi VM yang lebih cepat dan efisien

Micro-Hypervisor

Virtualization untuk embedded systems

Unikernel & Specialized VMs

OS minimalis untuk workload spesifik

GPU Virtualization

Akses GPU untuk workload komputasi tinggi

Real-Time Virtualization

Latensi rendah untuk aplikasi real-time

Security-Focused Virtualization

Isolasi keamanan yang lebih kuat

Masa Depan Virtualization

Edge Computing

Virtualization pada perangkat edge untuk latensi rendah

Quantum Computing

Virtualization pada sistem komputasi kuantum

AI-Driven Management

Manajemen resource otomatis menggunakan AI dan machine learning


Hybrid Cloud


Zero Trust

Konsep Dasar

- ✓ VM mensimulasikan komputer fisik dengan OS dan aplikasi sendiri
- ✓ Hypervisor mengelola virtualisasi dan resource allocation
- ✓ Berjalan di hardware fisik yang sama tanpa konflik

Jenis Hypervisor

Type 1 (Bare Metal)

Langsung di hardware, performa tinggi

Type 2 (Hosted)

Di atas OS host, mudah digunakan

Manfaat Utama

- ↗ Efisiensi penggunaan hardware
- 🛡 Isolasi antar sistem
- ↔ Portabilitas mudah
- ☁ Backup dan recovery cepat
- 📊 Konsolidasi server

Penggunaan

Cloud Computing

Desktop Virtualization

Testing Environment

Disaster Recovery

Development

Teknologi terus berkembang dengan penelitian baru

Networks & Distributed Systems / Advantages

Resource Sharing

- ≡ Berbagi data, file, dan informasi
- <> Aplikasi dan software bersama
- 📁 Hardware resources dapat digunakan bersama

Reliability & Scalability

Reliability

Tetap berjalan jika satu node gagal

Scalability

Mudah menambah kapasitas

Redundancy memastikan ketersediaan tinggi

Improved Performance

- ⚡ Parallel processing mempercepat kerja
- 🔄 Distribusi beban kerja ke banyak node
- ⚡ Skalabilitas kinerja yang tinggi

Flexibility & Economics

Flexibility

Konfigurasi dapat disesuaikan kebutuhan

Economic Benefits

Efisiensi biaya jangka panjang

👤 Komponen Jaringan

- 🖥️ **Nodes:** Komputer dan server
- 🔌 **Links:** Kabel dan wireless
- 📶 **Switch/Router:** Mengatur lalu lintas data

📊 Topologi Jaringan

Bus

Linear

Star

Centralized

Ring

Circular

Mesh

Fully Connected

Tree

Hierarchical

Hybrid

Combined

🌐 Jenis Jaringan

WAN

Global

Menghubungkan jaringan lokal secara global

MAN

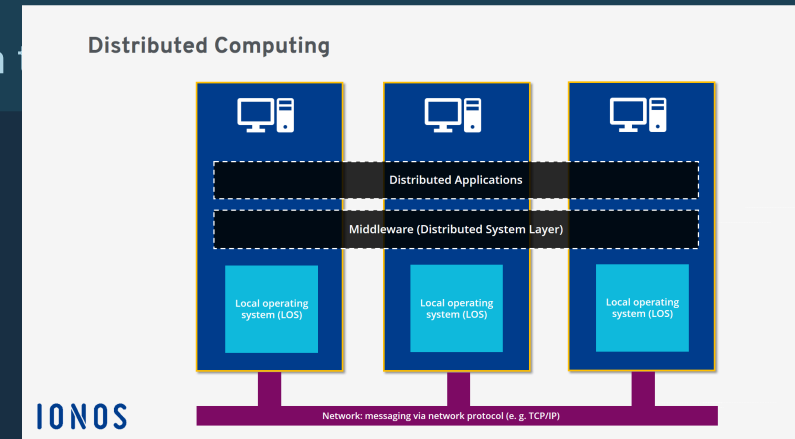
City

Menghubungkan area metropolitan/kota

LAN

Local

Area



Model Komunikasi

Client-Server

Client meminta layanan, server menyediakan jawaban

Peer-to-Peer (P2P)

Setiap node bisa bertindak sebagai client atau server

Message Passing

Mengirim pesan langsung antar proses dalam sistem

RPC (Remote Procedure Call)

Memanggil fungsi/prosedur yang berada di komputer lain

Protokol Komunikasi

TCP/IP

Reliable

Transmisi data yang andal dan connection-oriented

UDP

Fast

Cepat, connectionless, tanpa jaminan pengiriman

HTTP/HTTPS

Web

Protokol untuk akses web dan transfer data

SMTP/IMAP

Email

Protokol untuk pengiriman dan akses email

📶 Network OS

- ✓ Mengelola jaringan lokal dan koneksi
- ✓ Sharing file, printer, dan resources
- ✓ Komunikasi antar komputer dalam jaringan
- ✓ Setiap node punya OS sendiri

💻 Distributed OS

- ✓ Berjalan di beberapa node sebagai satu sistem
- ✓ Transparansi akses resource
- ✓ Load balancing otomatis
- ✓ Fault tolerance built-in

↔ Perbedaan Utama

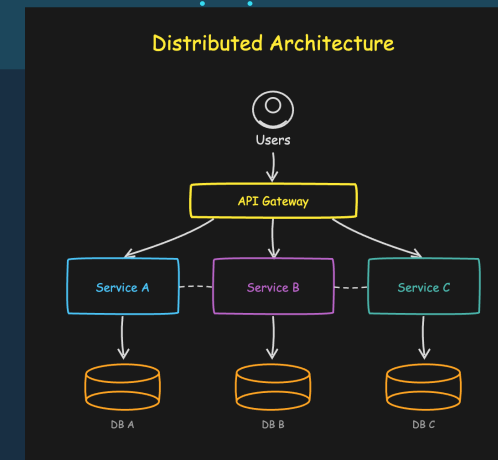
Network OS

OS jaringan biasa untuk koneksi lokal

Distributed OS

OS distribusi dengan transparansi penuh

Distributed OS menyembunyikan kompleksitas



Transparansi

Location

Pengguna tidak tahu lokasi data

Access

Akses resource seperti lokal

Failure

Pemisahan terhadap kegagalan

Keandalan

Redundancy

Backup di multiple node

Replication

Data tersalin di beberapa node

Fault Tolerance

Sistem tetap berjalan jika ada kegagalan

Performa

Latency

Waktu respon sistem

Bandwidth

Kapasitas transfer data

Throughput

Jumlah data per waktu

❦ Networks & Distributed Systems / Design Issues (Part 2)

🛡️ Keamanan

✅ Authentication

Verifikasi identitas pengguna

🔑 Authorization

Hak akses yang sesuai

🔒 Encryption

Enkripsi data sensitif

⚙️ Integrity

Keutuhan data terjamin

🔄 Consistency

📋 Strong

Data selalu sama di semua node

⌚ Eventual

Data akhirnya sama (waktu tertentu)

↔️ Conflict Resolution

Penyelesaian konflik data

📈 Consistency Models

Pilihan model sesuai kebutuhan

🔧 Fault Tolerance

🔍 Detection

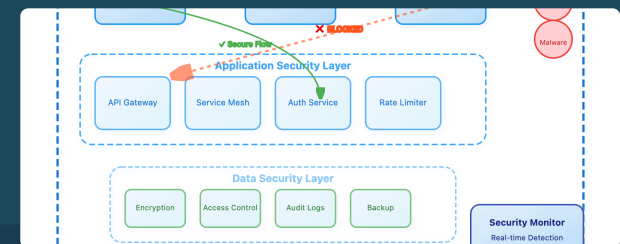
Deteksi kegagalan cepat

🕒 Recovery

Pemulihan otomatis setelah kegagalan

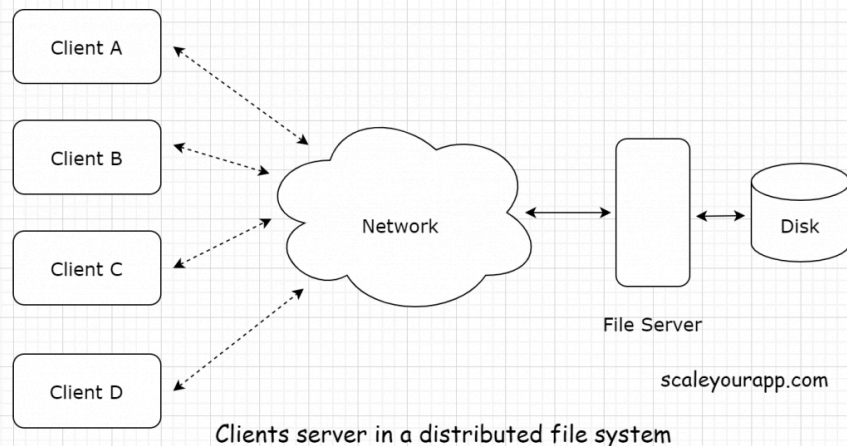
🛡️ Isolation

Isolasi kegagalan dari sistem utama



📁 Konsep DFS

- ☰ Sistem file tersebar di beberapa server
- 🖥️ Pengguna mengakses seolah-olah di lokal
- ⚙️ File management terpusat
- ✅ Availability tinggi dengan redundancy



📋 Kebutuhan DFS

- 📈 **Scalability**
Mudah ditambah kapasitas storage
- 🛡️ **Reliability**
Data tidak hilang dengan replication
- 🚀 **Performance**
Akses cepat dengan caching
- 👁️ **Transparency**
Lokasi file tidak diketahui pengguna
- 🔄 **Consistency**
Data konsisten di semua lokasi

Arsitektur DFS

Client

Meminta file ke server

Server

Menyediakan file untuk client

File System

Mengorganisir dan menyimpan file

Cache

Penyimpanan sementara file

Network

Komunikasi antar komponen

Komponen DFS

Directory Service

Manajemen direktori

File Server

Menyimpan file

Name Server

Translasi nama ke lokasi

Security Server

Otorisasi akses

Implementasi DFS

NFS

Network File System (Unix/Linux)

AFS

Andrew File System (CMU)

CIFS

Common Internet File System (Windows)

HDFS

Hadoop Distributed File System (Big Data)

Model Penamaan

Location Transparency

Nama file tidak menunjukkan lokasi fisik

Location Independence

File bisa dipindahkan tanpa mengubah nama

Naming Structure

Flat naming vs Hierarchical naming

Keuntungan

Sistem lebih fleksibel dan mudah dikelola

Transparansi Lokasi

- ✓ Client tidak tahu di mana file disimpan
- ✓ Server handle request dari mana saja
- ✓ Load balancing otomatis antar server

Transparansi Akses

- ✓ API yang sama untuk lokal dan remote
- ✓ Sistem operasi handle detail akses
- ✓ User experience konsisten di semua lokasi

Model Akses

Download Entire File

Seluruh file di-download dulu ke local

Upload Entire File

File di-upload setelah edit selesai

Partial Access

Hanya bagian file yang diakses saja

Streaming

Data diakses secara berurutan/stream

Caching

-  Local cache pada client untuk akses cepat
-  Cache consistency: sinkronisasi otomatis
-  Cache replacement saat memori penuh

Replication

-  **Primary Server:** Server utama untuk write
-  **Secondary Servers:** Server cadangan redundant
-  **Read Replicas:** Khusus untuk operasi read

Tantangan

Consistency Trade-offs

Kecepatan vs Konsistensi (CAP theorem)

Network Latency

Waktu tunda jaringan mempengaruhi performa

Security Concerns

Keamanan data saat ditransfer dan disimpan

Scalability Limits

Batasan dalam skala yang sangat besar

Masa Depan

Cloud-Native DFS

Dioptimalkan khusus untuk cloud computing

Edge Computing DFS

DFS yang berada di edge network

AI-Powered Optimization

Optimasi otomatis menggunakan Machine Learning

Blockchain-Based DFS

Keamanan dan integrity dengan blockchain

DFS adalah komponen kunci dalam distributed systems yang terus berkembang

Sistem Terdistribusi

- 🔗 Resource sharing antar node
- 📈 Skalabilitas dan flexibility
- 🛡️ Keandalan dan fault tolerance

Distributed File Systems

- ☁️ File management terpusat
- 🔒 Akses terdistribusi dengan transparansi
- 📄 Caching dan replication untuk performa

Komponen Utama

- 🏠 Nodes, links, network structure
- ↔️ Communication protocols (TCP/IP, UDP)
- 💻 Distributed OS dengan transparansi

Tantangan & Masa Depan

- ⚠️ Konsistensi, latensi, keamanan, skalabilitas
- 🚀 Edge computing, AI-driven, cloud-native

Distributed Systems adalah fondasi modern computing



Terima Kasih

Mata Kuliah Sistem Operasi - **Advanced Topics**

Telah membahas **Virtual Machines** dan **Networks & Distributed Systems** secara mendalam



Infrastructure Modern



Cloud Computing



Teknologi Masa Depan



Silakan Ajukan Pertanyaan



Buku Referensi

- Operating System Concepts - Silberschatz
- Distributed Systems: Principles and Paradigms - Tanenbaum
- Modern Operating Systems - Tanenbaum

Online Resources

- Microsoft Azure Documentation
- Google Cloud Platform Guides
- VMware Documentation
- Kubernetes & Docker Documentation

Jurnal Ilmiah

- IEEE Transactions on Computers
- ACM Transactions on Computer Systems
- Distributed Computing Systems Journals

Topik Penelitian

- Virtualization & Containerization
- Distributed Systems Architecture
- Cloud & Edge Computing



Dosen Pengampu: **Triawan Adi**
Cahyanto, M.Kom



Program Studi Teknik
Informatika



Universitas Muhammadiyah
Jember